

1. O que o VISÃO é agora (após a virada de 2026-08-09)

O projeto deixou de ser uma «rede de vários computadores» (enxame, recrutamento de voluntários, auto-replicação em rede). Passou a ser uma rede neural líquida de altas habilidades que roda inteiramente no seu notebook (Ryzen, 7,6 GB RAM, CPU) — e a busca continua sendo aprendizado contínuo rumo a AGI: uma rede que acumula conhecimento incrementalmente, barato, sem retreinamento global dispendioso e sem esquecer o que já aprendeu.

«**Atenção**» Redes fixas (toda CNN/RNN/transformer treinada uma vez) ficam presas a conhecimento obtido via treinamento caro e que não continua. A rede líquida do VISÃO **aprende incrementalmente**: cada nova experiência ajusta pesos locais, sem reprojetar tudo. É por isso que o projeto existe — e por que a parte de enxame era enfeite (e perigosa), não o núcleo.

«**Nota**» Corte de escopo (formalizado em `PLAN0.md` e `BACKLOG.json`, commit `030f0c3`): removidos **swarm multi-node, peer recruitment, self-replication over network, PoW spawn gate, remote quorum, DGM autonomous self-rewrite**. Mantidos: célula LTC, regras locais, aprendizado contínuo sem esquecer, estabilidade/Lyapunov, treino single-node, bundle autocontido, e R3 como «núcleo imutável do assistente local» (não anti-replicação).

2. Por que redes líquidas (e não fixas) são o caminho para AGI contínua

Uma rede neural tradicional guarda **todo** o conhecimento num único vetor de pesos globais. Treinar a tarefa B sobrescreve a representação da tarefa A → esquecimento catastrófico (McCloskey & Cohen, 1989).

Transformer piora: pesos congelados após deploy, backprop exige o grafo inteiro.

A célula líquida do VISÃO resolve isso por dois lados:

«**Matemática**» Cada neurônio segue a EDO (LTC, Hasani 2022):

$$\frac{dx}{dt} = -\left[\frac{1}{\tau} + f(x, u)\right](x - A), \quad (1)$$

onde $f = \sigma(W_{in}u + W_{rec}x + b) \in (0, 1)$ e A é potencial de reversão. A constante de tempo **efetiva** depende da entrada:

$$\tau_{ef} = \frac{\tau}{1 + \tau \cdot f}. \quad (2)$$

Entrada forte ($f \rightarrow 1$): τ_{ef} encolhe → reage rápido. Entrada fraca ($f \rightarrow 0$): $\tau_{ef} \rightarrow \tau$ → mantém memória de longo prazo (neuroplasticidade de curto prazo, em forma fechada). Memória **multiescala emergente** — sem «pastas» no código.

«**Nota**» O `cfc.py` do repositório implementa isto como **LTC com solver fundido** (Euler semi-implícito), não o Cfc fechado de Hasani (que interpola a trajetória por sigmoid sem passos). A estabilidade incondicional do solver (denominador sempre > 1) é o que permite usar dt grande sem explosão — técnica clássica de EDO rígida. Corrigido em `b9e8d5a`.

3. O que já é real e medido (não promessa)

Tudo abaixo roda e tem número no repositório:

«**Resultados empíricos**»

- **Aprendizado contínuo (Fase 0, `prototype/results_continual.json`, 5 seeds)**: esquecimento 97,2% menor que controle (média dos seeds; seeds 2–3 até **melhoraram** a tarefa A ao treinar B). Ablação $2 \times 2 \times 2$ mostra que o componente principal é **surpresa** ($-0,262$), não localidade (Oja $-0,092$); consolidação sozinha **piora** ($+0,178$).
- **Estabilidade (`results_stability.json`)**: $\rho(W_{rec}) = 0,7577 < 1$ (ESN viável), $\max|\lambda_J| = 0,9754 < 1$ (jacobiano contrativo), Lyapunov $\lambda = -0,7484$ (sub-caótico). **Segunda medição independente** via algoritmo de Benettin (1980) que eu rodei: $\lambda = -0,7876$. Convergência confirma o sinal.

- **Parâmetros (`results_param_count.json`):** CfC ≈ 5.002 params vs LSTM ≈ 67.850 ($13,56\times$ menor). Gap de acurácia em psMNIST: $-12,1$ pp (honesto; GRU 60k não medido).
- **FedAvg local (tarefa 2.1):** 2 processos, convergiu a $< 5\%$ do single-node. Agora serve só como referência de agregação local — sem rede.

«Atenção» O que o VISÃO não é: não é um LLM. Com 5 mil parâmetros ele brilha em **sequências contínuas, controle e aprendizado contínuo** — não em gerar texto como o GPT. Ele prevê dinâmicas, controla sistemas e nunca esquece; não redigirá prosa. Isso casa com seu plano de Computação Quântica (sistemas dinâmicos, Lyapunov — mesmo vocabulário).

4. O que muda no roteiro (nova Fase 5)

Em vez de coordenar nós, o esforço vai para aprendizado contínuo real no notebook:

«Nova Fase 5»

- **5.1** — Escalar a LTC de $n = 96$ para $n \approx 512\text{--}1024$ validando RAM/CPU do seu notebook (medir pico de RAM e tempo por passo).
- **5.2** — Substituir o brinquedo psMNIST por uma tarefa contínua real sua (ex.: previsão de série temporal da sua rotina/estudos) e demonstrar não-esquecer entre ≥ 2 tarefas sequenciais.
- **5.3** — Auto-ajuste **local** em sandbox (arquitetura/hiperparâmetros) — AutoML comum, com R3 protegendo o núcleo. Fora do escopo: replicação em rede.

As antigas tarefas 2.1–2.6 (Byzantinos, sharding, quorum de spawn, PoW) estão marcadas discontinued_2026_08_09 no BACKLOG.

5. Onde isto encosta no seu ITA

- **MAT-22 (sistemas dinâmicos):** a EDO da célula, estabilidade de Lyapunov e a «borda do caos» ($\rho(W) < 1$ é o **echo state property** de Jaeger 2001, não LTC; «líquido» vem das **Liquid State Machines** de Maass 2002) são exatamente o vocabulário que você verá.
- **Álgebra linear / MAT-12:** Oja (1982) é PCA online — o peso converge ao autovetor principal da covariância da entrada. É o mesmo motor de PCA em streaming.
- **Estatística experimental:** a ablação $2\times 2\times 2$ é **design de experimentos** puro.

6. Resumo para não perder o fio

VISÃO = uma rede líquida pequena, interpretável e barata que aprende **continuamente** no seu notebook, nunca esquece, e cujas decisões você entende. A parte de «exército de máquinas» foi cortada (segurança + realismo). O diferencial real — aprendizado contínuo minúsculo — está medido e funciona. Próximo passo de engenharia: escalar para o notebook e botar uma tarefa sua de verdade nela.

Fontes vivas: `visao/core/cfc.py`, `visao/governance/containment.py`, `prototype/results_continual.json`, `visao/analysis/results_stability.json`, `visao/bench/results_param_count.json`, `PLANO.md`, `BACKLOG.json`.